

Empirical Comparison of Single Probing and Double Probing in an Open-Addressing Hash Table under an LRU Workload

Will Meyer

Abstract

We compare two hash-table probing strategies, **single (linear)** and **double hashing**, under a Least Recently Used (LRU) workload.

All traces use a fixed seed and a timing harness that measures only the replay loop of operations performed on the open-addressed hash tables. Each implementation is evaluated under the same LRU trace so the comparison isolates the probing strategy itself.

Headline Finding

Even though theory says that double probing should outperform single probing in open addressing because its second hash step spreads items more evenly, in my measurements, I found the opposite to be true in certain metrics. Under this LRU workload, **single probing is consistently faster at large N**, despite having more probes.

Question and Hypotheses

Question:

Which method, **single (linear)** or **double probe hashing**, will perform better under an LRU (**least recently used**) stream (steady churn near capacity), and why?

Hypothesis:

I expect **Single** and **Double** probing to look similar at **small N**, and then **diverge as N grows**.

At **small N** (up to about 2^{16})

- Both methods should have short probe chains with low tombstone buildup, therefore, I expect similar elapsed times. Single probing may even be slightly faster early on due to its contiguous organization of data.
- Double probing's second hash function shouldn't matter much when the probe chains are short, so I do not think a major difference will be seen when N is small.

At **large N** (Near Capacity and LRU churn)

- I expect **Double probing to pull ahead**.
- Theoretically, linear probing tends to form long **primary clusters**, especially under an LRU where the same regions become more impacted.
- Double probing should spread data more evenly in the table and should maintain shorter probe sequences and lower average_probes.
- With fewer long searches, its overall throughput should be higher and latency lower once the table is large.

In this experiment, we will test this hypothesis by looking at time curves, work-per-op metrics, and table structure plots (run-length 0/1 maps and histograms).

Method

This experiment evaluates a hash table under an **LRU-Profile**. The goal is to mimic how an OS manages a limited set of pages in memory. (recently accessed items can remain, but the **least-recently-used** item gets evicted when space runs out)

For each table size **N**

1. The harness generates a stream of LRU accesses.
2. Emit a csv with metrics based on a replay of the same hits and evictions from the trace.

LRU Rules

- **If a key is already present** → count a hit and move it to MRU.
- **If it's absent and the table isn't full** → insert it and mark it MRU.
- **If it's absent and the table is full** → evict the LRU key, then insert the new one.

Trace Opcodes

- **I key** — access the key and on a miss, insert it (or evict first if needed).
- **E key** — evict this key (appears only when full just before an insert).

This process allows us to replay the exact sequence of an LRU algorithm.

Dataset Summary

All keys come from `20980712_uniq_words.txt`, a ~370 MB file with about 20×2^{20} unique tokens pulled from IMDB reviews and Newsgroup text.

For the LRU profile, I used the provided trace files and the full collection of traces covering $N=2^{10}$ to 2^{20} . These files encode the access stream for the LRU workload, which is how my harness could perform proper measurements and provide the necessary statistics.

Profile Summary (LRU)

Parameter	Value
Profile	LRU profile
Range	N from 2^{10} to 2^{20}
N values	1024 up to 1,048,576 (powers of two)
Seeds	Single fixed seed (23)

3.5 CSV Fields

Column	Meaning
impl	hash map implementation used
profile	workload name
trace_path	path to the trace file
N	dataset size for the trace
seed	RNG seed
elapsed_ms	total replay time
ops_total	total operations processed
inserts	number of insert operations
erases	number of erase operations
table_size	table capacity
active	number of ACTIVE slots
available	number of AVAILABLE slots
tombstones	number of DELETED slots
total_probes	total probe steps taken
deletes	delete operations
lookups	lookup operations
full_scans	count of full-table scans
compactions	compactions triggered
max_in_table	max ACTIVE entries seen
available_pct	percent AVAILABLE
load_factor_pct	percent ACTIVE of table_size
eff_load_factor_pct	ACTIVE out of (ACTIVE + AVAILABLE)

Column	Meaning
tombstones_pct	percent DELETED of table_size
average_probes	average probes per operation
probe_type	single or double hashing
compaction_state	compaction on/off

Results & Analysis

Each subsection below corresponds directly to required questions in Sections 5.1 and 5.2 of the brief.

Expectations vs. Observations (5.1)

Q1. Expected ordering (Single vs. Double) at small N and large N

Small N:

Single probing should outperform because its probe paths are short and data is contiguous which is better for cache locality.

Large N:

Double probing should provide better performance because it should eliminate long primary clusters and keep probe lengths smaller.

Q2. Where the data agrees or disagrees

Figure A: Elapsed time (ms) vs. N for single and double probing implementations.



After plotting **elapsed time vs. N** , the data matched some expectations but not all.

- At **small N** , single and double hashing look nearly the same.
- As N grows (especially **2^{15} – 2^{18}**), the curves start to separate.
- The **$N \log N$ line** (dotted) serves as a sanity check that this chart reflects the behavior of a real hash table. Both SINGLE and DOUBLE stay below that line across all N , which is what we expect when operations are roughly **$O(1)$ amortized**.

Since the data looks correct, the differences come down to other factors. I expected double probing to eventually win in performance because it reduces primary clustering, but the opposite happened. Double's extra hash step requires more computation and is less efficient for memory access, so even though it has shorter probe chains, SINGLE ends up faster at large N because its probes require less arithmetic and are more cache friendly.

Q3. One N where separation is largest (quantified difference)

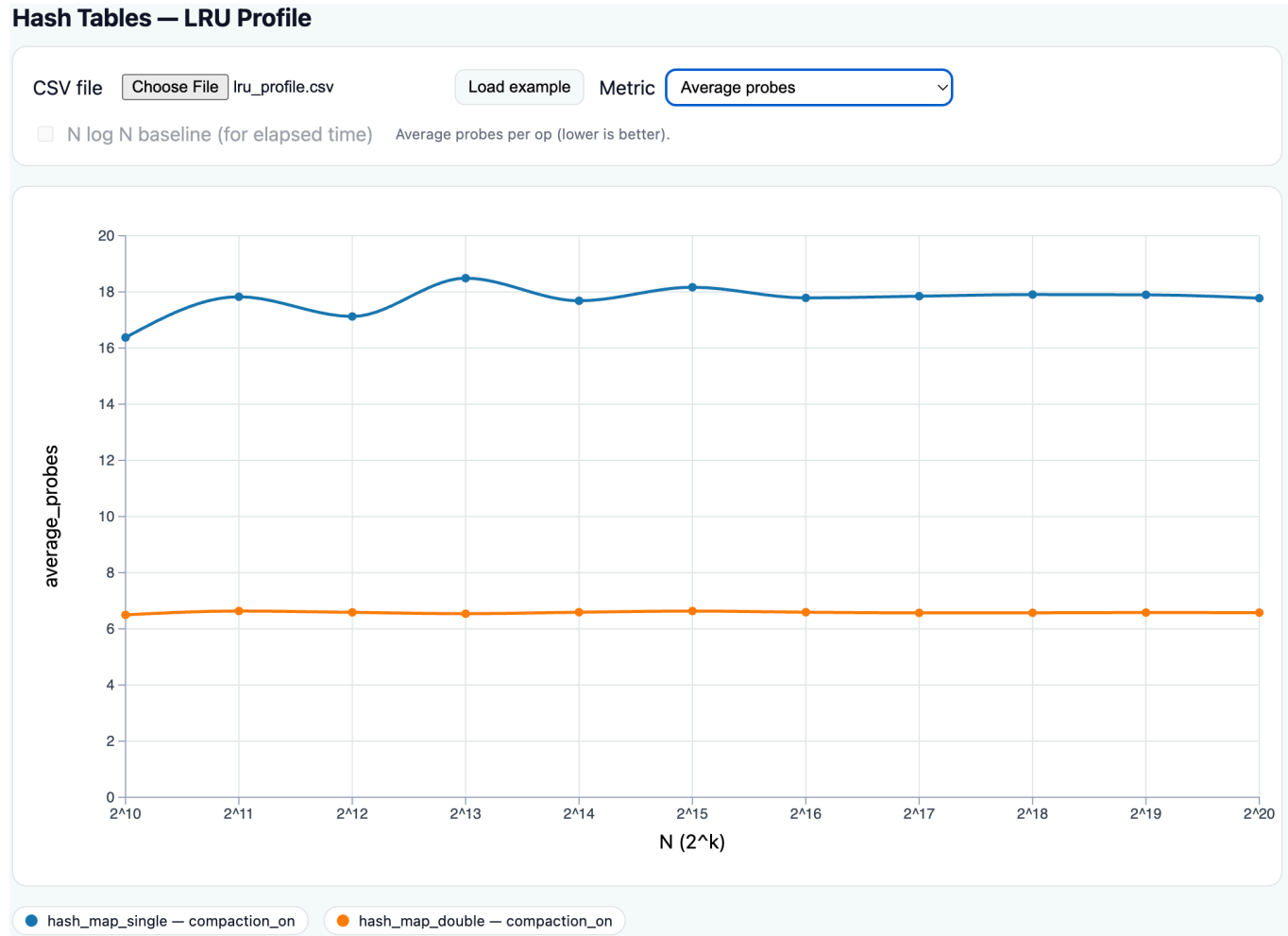
- By $N = 2^{20}$, single probing is actually about 30% faster than double.

4.2 Work per Operation vs Wall-Clock Time (5.1.2)

Q1. Do fewer probes consistently correlate with lower time across all runs?

No, in these runs, fewer probes do not guarantee a lower elapsed time across almost all N .

Figure B: Average Probes vs. N for single and double probing implementations.



Q2. Find any N where the ordering by average_probes and by time disagree. Look at other metrics (e.g., hashing overhead, occupancy, compactions) to explain the mismatch

From about $N = 2^{16}$ onward, the ordering disagrees.

For example:

- **Double probing** has much lower average_probes (around 6.5) but a higher elapsed_ms.
- **Single probing** has closer to 16-18 average_probes and a lower elapsed_ms.

Hashing Overhead has higher computation cost

Double probing does a second hash for every miss and can also on each probe. Even if the probe count is lower, the computation cost per probe is higher due to the added arithmetic.

Occupancy and Cache Behavior

Single probing forms long contiguous clusters which is normally a disadvantage. That said, the added benefit of improved cache locality can not be understated, and the data agrees. Double probing has more work to do, jumping around the table substantially more, which causes cache misses and ultimately, lower performance in terms of `elapsed_ms`.

Compactions

Compactions dramatically reduce tombstones, but the process itself is more expensive. If Double hashing triggers a compaction at an N value where Single does not, the timed performance of Double hashing will drastically fall.

Hashing Cost and Memory Locality (5.1.3)

Q1. Per-operation hashing cost differences and where constant factors matter most

In my implementation:

- **Single probing** does one hash per operation.
- **Double probing** does two hashes per operation.

I would expect that to matter more at **small N**, but the data I collected shows something a bit different. Double has lower probe counts even at the smallest N, but this advantage never seems to allow it to overtake Single. This means that the extra hash along with the extra math overhead does not pay off in the long run at all N.

Q2. Look for locality effects (where contiguous accesses help single probing)

As seen in Figure B above (*Average Probes vs. N*), I never see a region where the probe counts are close and single clearly pulls ahead in terms of `elapsed_ms`.

What I do see is that even with worse probe counts, single still wins in time at large N, which is a strong indication that contiguous access and fewer hash steps beat the reduction in primary clustering that Double Probing delivers.

Q.3 Where does the more “de-clustered” method start to pull ahead?

In theory, double hashing should pull ahead at large N, when shorter clusters matter more.

In my experiment:

- Double wins on **probe count** at **all** N.
- It **never** really wins on **elapsed time** at large N.

So the “de-clustered” method **wins in average probe length**, but **loses in elapsed_ms** throughout all ranges of N.

Compaction Effects (5.1.4)

****Q1. Across N, which method triggers more compactions, and does it align with tombstones_pct or higher eff_load_factor_pct before compaction?**

As expected, across every N in the LRU trace, **Double probing consistently triggers more compactions than Single probing**. (See *Figure C: Compactions (count) vs N*)

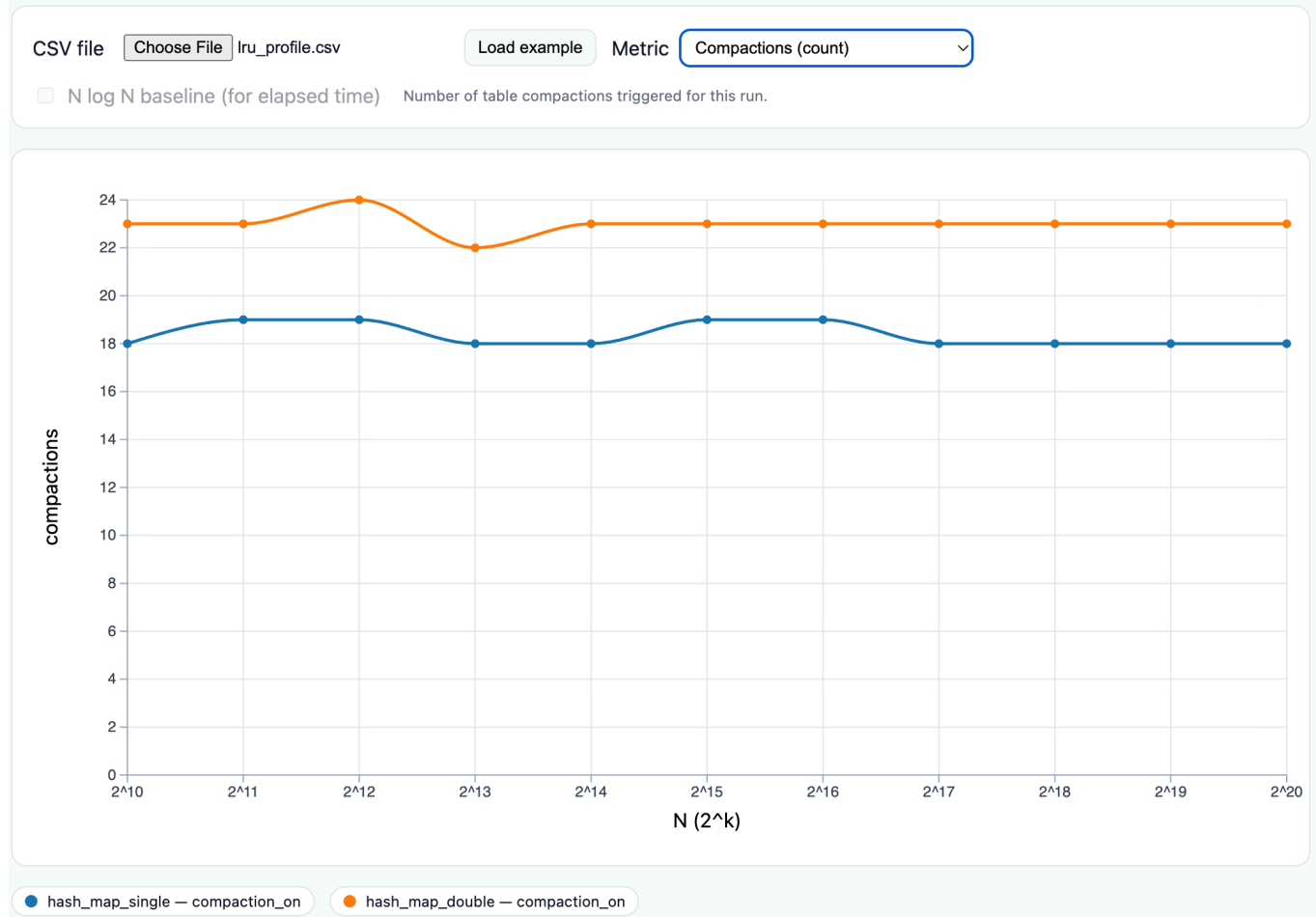
- Single stays around 18-19 compactions per run.
- Double stays around 22-24. This is almost a 30% increase, on average,

This lines up with expected behavior and means that:

- Single lets tombstones and clusters accumulate.
- Double compacts more aggressively and keeps tombstones and effective load to a minimum.

Figure C: Compactions (count) vs. N for single and double probing implementations.

Hash Tables — LRU Profile



Q2. At same N , do more compactions correlate with higher elapsed time? Give example.

Yes, especially at larger N .

Example: $N = 2^{17}$ (and similarly above it)

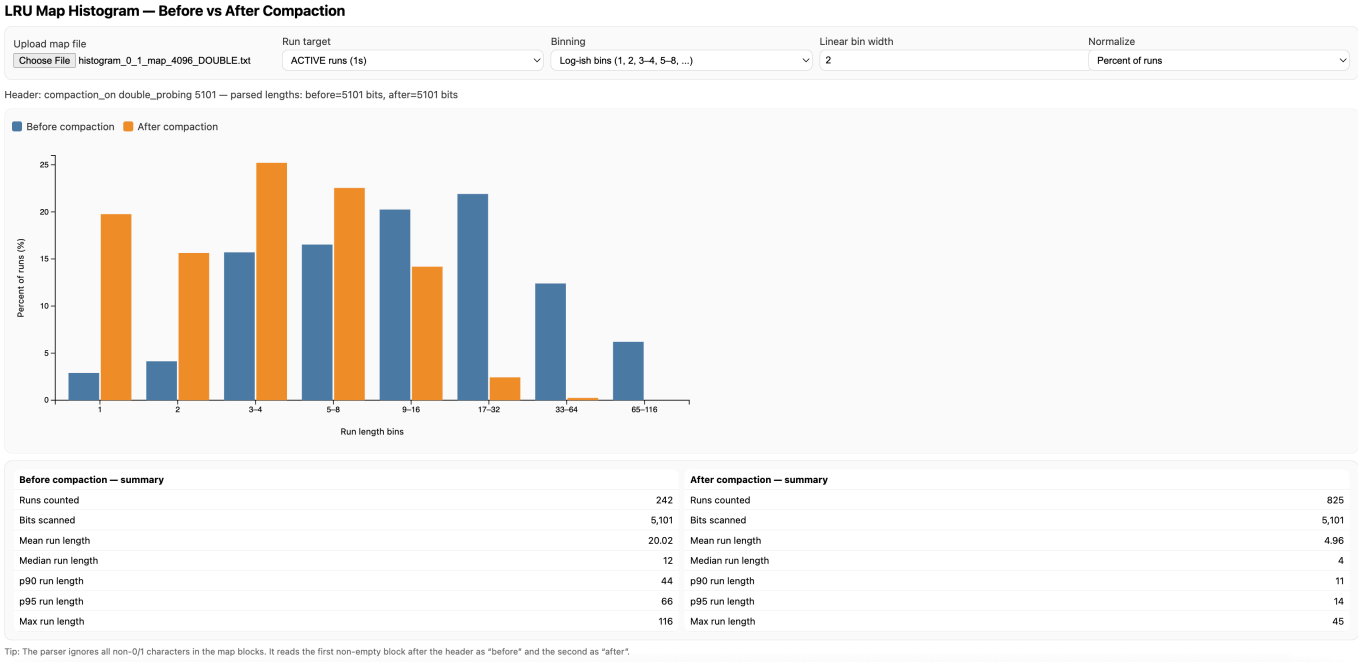
- Double triggered **more compactions** (e.g., 23) and had **higher elapsed time** (e.g., 2102.40 ms).
- Single triggered **fewer compactions** (e.g., 18) and had **lower elapsed time** (e.g., 1874.33 ms)

This makes sense because each compaction walks the entire table. At big N , those passes are computationally expensive enough that a few extra compactions can wipe out the benefit of shorter probe chains.

Q3. Use before/after maps: do you see a structural change that would plausibly explain changes in average_probes?

To answer whether compaction meaningfully changed the table structure, I used the run-length histogram summaries generated by the standalone mapping tool for **both Single and Double probing** on the same trace (`lru_profile_N_4096_S_23.trace`).

Figure D. Active-run histogram for DOUBLE hashing (N = 4096)



Double Probing — Before vs After Compaction

Before the final compaction, Double shows:

- **Mean run = 20.02**
- **p90 = 44**
- **p95 = 66**
- **Max run = 116**

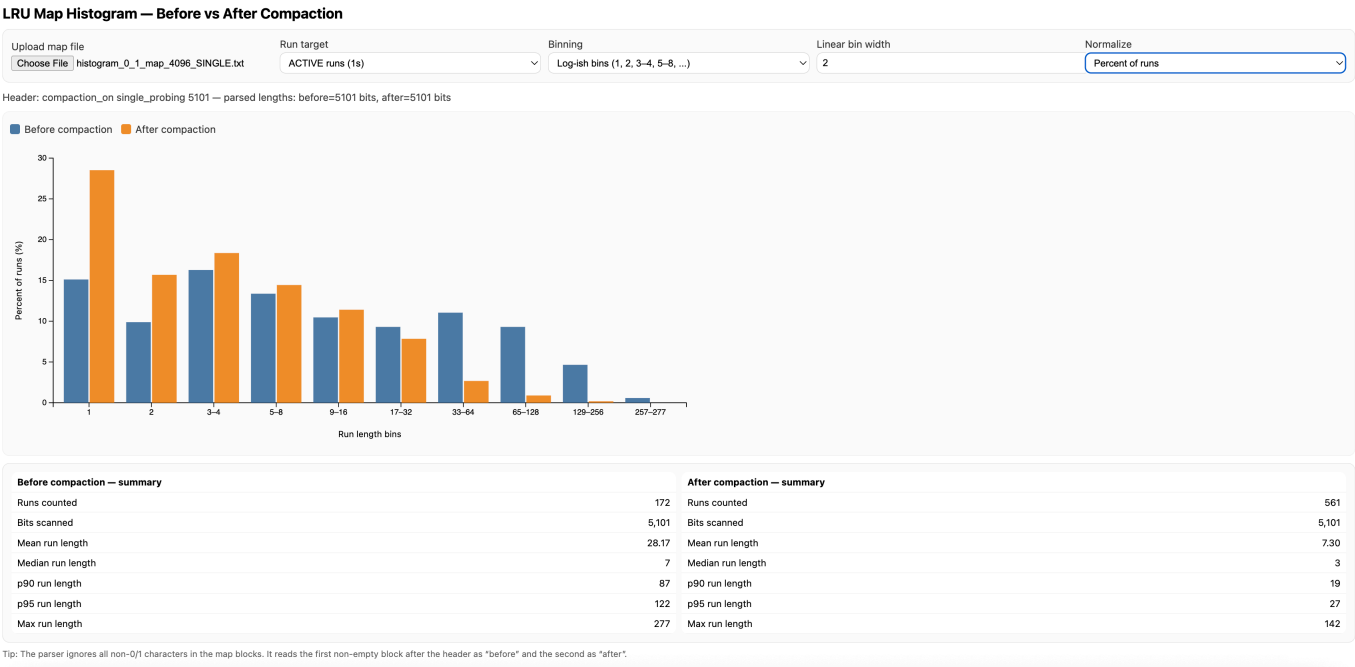
After compaction, the structure changes dramatically:

- **Mean run drops to 4.96**
- **p90 drops from 44 → 11**
- **p95 drops from 66 → 14**
- **Max run drops from 116 → 45**

This is a significant reduction in cluster length. The number of runs also jumps from 242 → 825, meaning compaction broke long clusters into many smaller segments.

This aligns with **Double's** much **lower average_probes count**, and it also shows how Search would benefit because there wouldn't be long traversals of probe chains.

Figure E. Active-run histogram for SINGLE hashing (N = 4096)



Single Probing — Before vs After Compaction

Single probing shows the "extreme" before picture:

- **Mean run = 28.17**
- **p90 = 87**
- **p95 = 122**
- **Max run = 277**

After compaction, Single does improve:

- **Mean run drops to 7.30**
- **p90 drops from 87 → 19**
- **p95 drops from 122 → 27**
- **Max run drops from 277 → 142**

Compaction helps Single significantly, but even after compaction, it still has significantly larger clusters than Double. A max run of 142 after compaction is still bigger than Double's worst cluster (116) before compaction. This explains why Single continues to have higher **average_probes** and would suffer adverse effects in `Search`.

Final Interpretation

The run-length statistics line up with expectations, and exactly with the probe counts. **Double probing** breaks up clusters a lot, turning long runs into short, easily traversable clusters. **Single probing** compacts less often and leaves larger clusters intact.

In an actual system, shorter run lengths after compaction directly translate into faster lookups, inserts, and deletions.

Throughput & Latency Cross-Check (5.1.5)

Q1. Do throughput and latency tell same story as elapsed time? If not, where do they diverge?

Throughput and latency mostly tell the same story as elapsed time, but the charts are inverted.

- **Latency (ms/op)** for DOUBLE is **always above** SINGLE and bends upward starting around $N=2^{18}$.
- **Throughput (ops/ms)** shows the reverse: DOUBLE is **always below** SINGLE and drops noticeably around $N=2^{18}$.

Because the trace has the same `ops_total` for both methods, these metrics are just different ways of showing elapsed time. The interesting difference is where the curves change behavior, around $N=2^{18}$. where DOUBLE's latency starts climbing and its throughput falls in kind.

Figure F: Throughput (ops/ms) vs. N for single and double probing implementations.

Hash Tables — LRU Profile

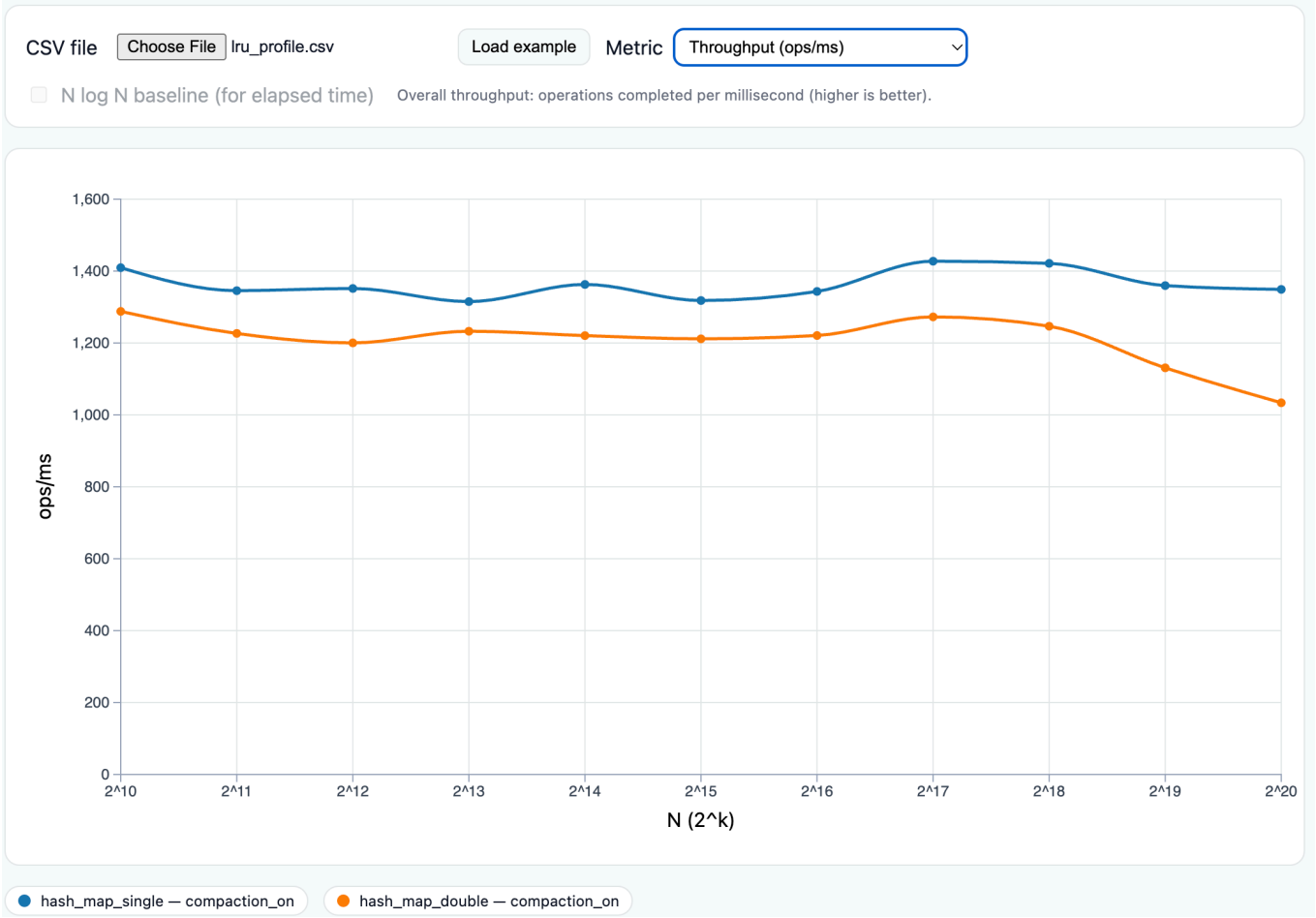
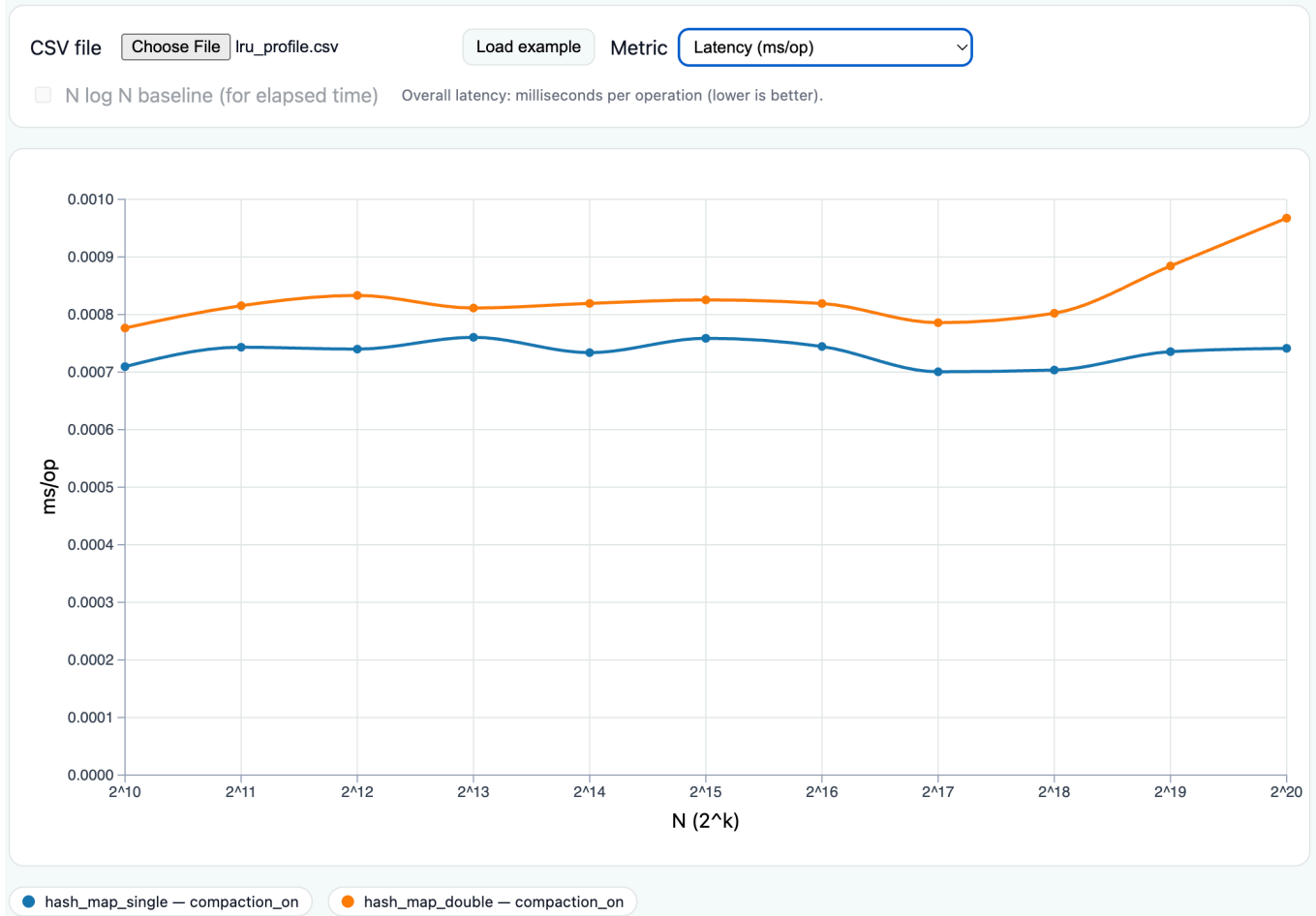


Figure G: Latency (ms/ops) vs. N for single and double probing implementations.

Hash Tables — LRU Profile



Q2. Which metric makes single vs. double separation clearer at large N? Why?

For me, **latency (ms/op)** makes the separation at large N the clearest since it directly shows the cost in terms of time per operation. At **large N** (e.g., 2^{18}), double's latency curve begins to break away from single's and the gap keeps expanding through $N=2^{20}$.

Since throughput is an inverted expression, it tells the same story, but the charts are flipped.

Occupancy Sanity Check (5.1.6)

Q1. For each N, do load_factor_pct, eff_load_factor_pct, tombstones_pct change much?

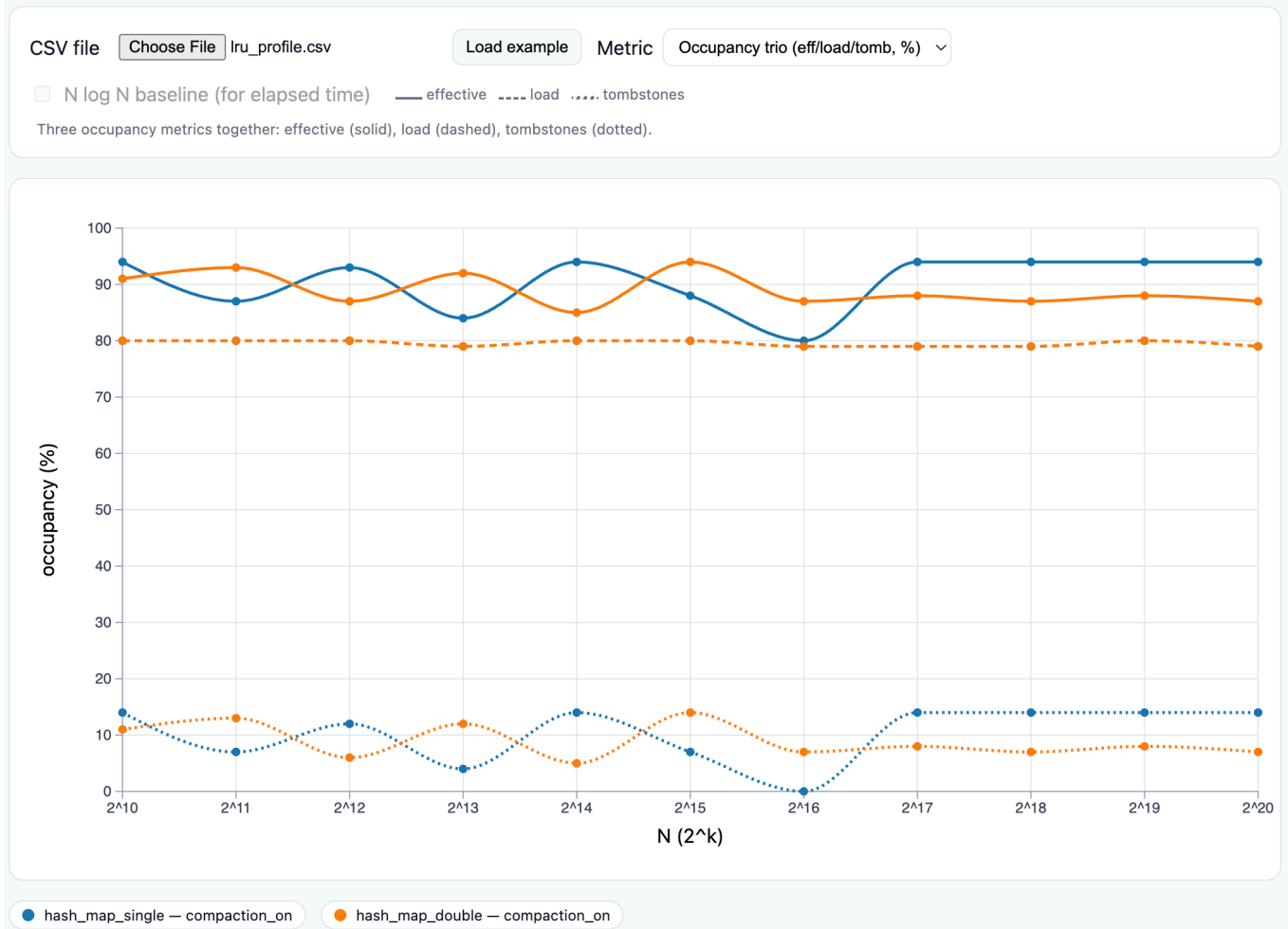
No, they stay basically flat across N for both SINGLE and DOUBLE.

- The **load factor** remains around ~80% the whole time for both methods.

- The **effective load** and **tombstone %** oscillate around each other until $N=2^{18}$, and then stay fairly constant up through $N=2^{20}$.

Figure H: Occupancy trio (eff/load/tomb, %) vs. N for single and double probing implementations.

Hash Tables — LRU Profile



Q2. At N where time gap is largest, compare the occupancy metrics

The elapsed time gap is largest at $N = 2^{20}$, (as seen in Figure A above).

Occupancy Metrics at that level:

- **Effective load** is **94% for SINGLE** vs **87% for DOUBLE**. (See top 2 lines, orange = Double blue = Single, Figure H)
- **Load factor** is the same for both: **79%** (SINGLE and DOUBLE). (See middle orange line Figure H)
- **Tombstones %** is **14% for SINGLE** vs **7% for DOUBLE**. (See bottom 2 lines, orange = Double blue = Single, Figure H)

So SINGLE is actually operating with a higher effective load and *more* tombstones than DOUBLE, even though they share the same 79% load factor, and Single still finishes faster.

Q3. Can these occupancy differences explain the time gap?

Not really, in fact, they point the other way.

At $N = 2^{20}$, SINGLE has **higher effective load** and **more tombstones**, which should make its probes more expensive than DOUBLE's. But in the timing data (See *Figure A: elapsed_ms vs. N*), **SINGLE is still faster** at that N. That means the time gap **cannot be explained by occupancy alone**. Instead, it has to come from the cost of computation per probe as well as cache line efficiency.

Before/After Compaction Structure (one snapshot) (5.1.7)

Capture ACTIVE/DELETED maps before and after compaction

For $N = 4096$, I used before/after compaction maps for both Single and Double, and built run length histograms

(_ See above for Figure D: Double, Figure E: Single).

Build run-length histograms for snapshots; report mean, median, p95, max

(Following Statistics from Figure D: Double, Figure E: Single) -- See above

DOUBLE hashing ($N = 4096$)

Metric	Before	After
Runs counted	242	825
Mean run length	20.02	4.96
Median run length	12	4
p90	44	11
p95	66	14
Max	116	45

SINGLE hashing ($N = 4096$)

Metric	Before	After
Runs counted	172	561
Mean run length	28.17	7.30
Median run length	7	3

Metric	Before	After
p90	87	19
p95	122	27
Max	277	142

Q1. How did distributions shift and how does that relate to average_probes?

From the afore mentioned histograms, we see that compaction shortens long runs and increases short runs. This skews the tables left and effectively brings the average number of probes down.

- SINGLE p95: **122** → **27**, meaning 10% of runs are now 27 or longer
- DOUBLE p95: **66** → **14**, meaning 10% of runs are now 14 or longer
- Max run length saw significant reduction (e.g., DOUBLE **116** → **45**)
- Means dropped by **4–6×**

Both methods benefit, as before compaction, probes must walk long stretches of ACTIVE or DELETED cells. After compaction, probe lengths shrink, but **double** ends up with many shorter distributions of runs than **single**. That matches expectations given that Double consistently has lower `average_probes` .

Maps + Histograms (5.2.4) -- Addl. Questions to answer

Q1. Where are the longest 1-runs—few big blocks or many medium ones?

Both methods have **a few big blocks of 1's** before compaction, not a collection of medium ones. Meaning, long ACTIVE+DELETED clusters that stretch across big sections of the table.

- SINGLE: max run = **277**, p95 ≈ **122**
- DOUBLE: max run = **116**, p95 ≈ **66**

Q2. At same N, do single and double probing show visibly different clustering?

Yes. At the same N and before compaction:

- **SINGLE** probing shows **heavier clustering** which means fewer runs overall (**172** runs) but much longer in length.
 - **DOUBLE** probing has **more, shorter runs** (**242** runs), but more broken up.
-

Q3. Does compaction break up long 1-runs?

Yes, very clearly.

Comparing the **before** vs **after** snapshots:

- **DOUBLE**: max run goes from **116** → **45**, mean from **20.02** → **4.96**, runs increase **242** → **825**.
- **SINGLE**: max run goes from **277** → **142**, mean from **28.17** → **7.30**, runs increase **172** → **561**.

Run counts increase significantly for both which is expected behavior as it clear tombstones and rebuilds the table.

Q4. After compaction, do long runs shrink? Do more short runs appear?

Yes to both.

- **SINGLE** shifts from being more spread out, to the table being skewing left, ~30% of run length 1.
- **DOUBLE** shifts from being more skewed right, towards the **3–4** run length range, a ~25% of runs in that range.

So compaction literally **reshapes the whole distribution** toward many more short runs for both methods.

Q5. At same N, which has more long runs: single or double?

SINGLE probing clearly has more long runs.

- **Before compaction**: **SINGLE** max = **277**, p95 = **122** vs **DOUBLE** max = **116**, p95 = **66**.
- **After compaction**: **SINGLE** max = **142**, p95 = **27** vs **DOUBLE** max = **45**, p95 = **14**.

SINGLE's histogram still stretches farther to the right, even after compaction, so SINGLE maintains larger clusters than DOUBLE.

Q6. When you see longer runs, do you also see higher avg_probes and elapsed_ms?

In my data, longer runs equates to higher *average_probes*, but not necessarily higher *elapsed_ms*.

(_ See Figure A: *elapsed_ms* vs Figure B: *avg_probes*)

Single probing, for example, has longer runs and higher probe counts, but it still finishes faster at large N because probes traverse the table contiguously and it only has one hash step. In practice, this beats double probing which has lower *average_probes* , but higher *elapsed_ms* .

Q7. When compaction reduces long runs, do probes & time improve?

Yes. Compaction refreshes the table and so the few long runs of 1's in the map become many short runs, so probes reach an empty slot much sooner. That reduces both the average and worst case probe lengths.

Lower probe counts should translate into better *elapsed_ms* , since the table isn't requiring such long traverses through clusters of ACTIVE/DELETED regions anymore. The real world result would be felt in the operations: Insert, Search and Delete .

Q8. How do load_factor_pct, eff_load_factor_pct, and tombstones_pct line up with maps? **

The *load_factor_pct* stays around ~80% for both methods, so by itself it doesn't explain clustering or time.

At large N, SINGLE has a **higher eff_load_factor_pct** and more tombstones (e.g., *eff_load_factor_pct* is about **94%** and *tombstones* is about **14%** vs DOUBLE at about **87%** and **7%** tombstones).

That means a higher fraction of the table is either ACTIVE or DELETED), which matches SINGLE's longer runs lengths and worse probe counts.

After compaction, **tombstones_pct drops** and **eff_load_factor_pct moves closer to load_factor_pct**. This is consistent with fewer tombstones and shorter runs in the histograms

In short, The **Maps & histograms** show equally how clustered the table is as a result of the **eff_load_factor_pct and tombstones**. In the end, the performance cost shows up in the number of probes and ultimately, the **elapsed_ms** per operation.

5. Discussion

The goal of this experiment was to keep the table in a highly clustered state, constantly cycling the same words in and out under LRU. From this, there are two consequences to point out:

1. **Clustering is a consistent problem.**

Both SINGLE and DOUBLE probing accumulate long runs of ACTIVE+DELETED over time. Compaction helps by breaking up those clusters, but the LRU workload will always return the table to a full state, particularly with tombstones, which is bad for operations.

2. **At large N, the hashing cost and cache locality matter more than occupancy.**

Structurally, **single probing** traverses the table in a straight line: index, index+1, index+2... which matches cache behavior.

Double hashing, however, deliberately jumps around the table by adding a second hash step. That breaks up primary clusters, but it also means more random access and each probe is more likely to land in a different cache line, so you pay more cache misses per operation. This difference is what turns fewer probes into slower overall time for double hashing.

So under this LRU workload, we can say that probe count, hashing cost and memory locality are the biggest contributors to runtime and compaction only temporarily improves the structure before the table gets full again.

6. Takeaways

Even though theory says DOUBLE hashing should perform better at high occupancy, the measurements captured from this LRU workload shows the opposite.

- **Double probing** consistently has fewer probes but triggers more compactions, pays extra computational cost for the second hash, and ends up slower at large N.
- **Single probing** has longer probe chains and heavier clustering, but it does fewer compactions and benefits from contiguous memory access and less arithmetic. That led to faster times in the end.

In real systems, double hashing is still considered the “gold standard” for open addressing because it avoids primary clustering and would typically give better performance for `Insert` , `Search` , and `Delete` . This experiment doesn't negate that, but it does show that under this LRU workload, constant factors and cache locality outweighed the theoretical advantages of double hashing in terms of raw timing benchmarks.